



Báo cáo Cloud - sdgg

điện toán đám mây (Trường Đại học Công nghệ Thông tin và Truyền thông Việt - Hàn)



messages.pdf_cover_qr_code_label

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
VÀ TRUYỀN THÔNG VIỆT - HÀN
KHOA KHOA HỌC MÁY TÍNH

-----□□□□-----



ĐỒ ÁN MÔN HỌC
ĐIỆN TOÁN Đám Mây

**Nghiên cứu mô hình cung cấp tài nguyên trong điện
toán đám mây - Case Study xây dựng máy chủ ảo
Private với mã nguồn mở Open Stack**

Sinh viên thực hiện: **Nguyễn Văn Bình** – 22IT028
 Phạm Văn Duy – 22IT053
 Trương Công Tú – 22IT323
 Lê Văn Nhật Hoàng – 22IT096

Giảng viên hướng dẫn: **TS. Nguyễn Hà Huy Cường**

Đà Nẵng, tháng 12 năm 2025

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
VÀ TRUYỀN THÔNG VIỆT - HÀN
KHOA KHOA HỌC MÁY TÍNH

-----□□□□-----



ĐỒ ÁN MÔN HỌC
ĐIỆN TOÁN ĐÁM MÂY

**Nghiên cứu mô hình cung cấp tài nguyên trong điện
toán đám mây - Case Study xây dựng máy chủ ảo
Private với mã nguồn mở Open Stack**

Sinh viên thực hiện: **Nguyễn Văn Bình** – 22IT028
 Phạm Văn Duy – 22IT053
 Trương Công Tú – 22IT323
 Lê Văn Nhật Hoàng – 22IT096

Giảng viên hướng dẫn: **TS. Nguyễn Hà Huy Cường**

Đà Nẵng, tháng 12 năm 2025

LỜI CẢM ƠN

Chúng xin gửi lời cảm ơn chân thành đến quý thầy cô của Trường Đại học Công nghệ Thông tin và Truyền thông Việt - Hàn vì đã giảng dạy và truyền đạt những kiến thức quý báu. Sự tận tâm và nhiệt huyết của thầy cô đã tạo điều kiện cho chúng em thực hiện báo cáo này một cách tốt nhất.

Đặc biệt, chúng em xin gửi lời cảm ơn sâu sắc đến thầy Cường đã luôn tận tình chỉ dẫn, truyền đạt kiến thức trong suốt quá trình thực hiện đồ án môn học này. Dù đã nỗ lực hết mình, chúng em hiểu rằng báo cáo này vẫn còn nhiều thiếu sót. Chúng em rất mong nhận được sự thông cảm và chỉ dẫn tận tình từ quý thầy cô.

Chúng em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

Chữ ký của GVHD
(ký ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN.....	i
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN.....	ii
MỤC LỤC.....	iii
DANH MỤC HÌNH ẢNH.....	v
MỞ ĐẦU.....	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu của đề tài.....	1
3. Đối tượng và phạm vi nghiên cứu.....	1
4. Phương pháp nghiên cứu.....	2
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	3
1.1. Tổng quan về Điện toán đám mây (Cloud Computing).....	3
1.1.1. Định nghĩa và đặc điểm.....	3
1.1.2. Các mô hình dịch vụ (IaaS, PaaS, SaaS).....	3
1.1.3. Các mô hình triển khai (Public, Private, Hybrid Cloud).....	4
1.2. Giới thiệu về mã nguồn mở OpenStack.....	5
1.2.1. Lịch sử hình thành và phiên bản 2024.1 (Caracal).....	5
1.2.2. Các thành phần cốt lõi.....	6
1.2.3. Cơ chế hoạt động và tương tác giữa các Service.....	6
1.3. Công nghệ Container và Tự động hóa trong OpenStack.....	7
1.3.1. Docker và Containerization.....	7
1.3.2. Ansible – Công cụ quản lý cấu hình.....	8
1.3.3. Kolla-Ansible: Ưu điểm so với các phương pháp cài đặt truyền thống.....	8
CHƯƠNG 2: THIẾT KẾ VÀ XÂY DỰNG.....	10
2.1. Phân tích yêu cầu hệ thống.....	10
2.2. Mô hình kiến trúc triển khai.....	11
2.2.1. Sơ đồ logic hệ thống.....	11
2.2.2. Quy hoạch địa chỉ IP và Network Topology.....	12
2.2.3. Đánh giá mô hình kiến trúc Multi-node.....	13
2.3. Chuẩn bị hạ tầng ảo hóa.....	13
2.3.1. Cấu hình Máy Host.....	13
2.3.2. Thiết lập KVM và mạng ảo.....	14
2.4. Xây dựng các Node thành phần.....	14

CHƯƠNG 3: TRIỂN KHAI VÀ THỬ NGHIỆM.....	16
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	17
TÀI LIỆU THAM KHẢO.....	18

DANH MỤC HÌNH ẢNH

No table of figures entries found.

MỞ ĐẦU

1. Lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số, Điện toán đám mây (Cloud Computing) đã trở thành hạ tầng cốt lõi cho mọi hoạt động công nghệ thông tin. Sự bùng nổ của dữ liệu và nhu cầu về tính linh hoạt trong việc cấp phát tài nguyên đã khiến các doanh nghiệp dần chuyển dịch từ hạ tầng vật lý truyền thống sang các mô hình đám mây.

Tuy nhiên, việc sử dụng Public Cloud (như AWS, Azure, hay Google Cloud) đôi khi khiến doanh nghiệp gặp thách thức về chi phí duy trì dài hạn, quyền kiểm soát dữ liệu nhạy cảm và tính tuân thủ pháp lý. Chính vì vậy, mô hình Private Cloud (Đám mây dùng riêng) nổi lên như một giải pháp tối ưu, cho phép tổ chức sở hữu toàn quyền hạ tầng, tối ưu hóa hiệu suất và bảo mật thông tin tuyệt đối.

Trong số các giải pháp xây dựng Private Cloud, OpenStack là nền tảng mã nguồn mở phổ biến và mạnh mẽ nhất thế giới hiện nay. Với phiên bản mới nhất 2024.1 (Caracal), OpenStack không chỉ cung cấp khả năng quản trị tài nguyên linh hoạt mà còn hỗ trợ tốt các công nghệ hiện đại như Containerization và AI. Việc triển khai OpenStack bằng công cụ Kolla-Ansible giúp đơn giản hóa quá trình vận hành thông qua việc "Container hóa" các dịch vụ, đảm bảo tính ổn định và dễ dàng nâng cấp.

Xuất phát từ thực tiễn đó, tôi/nhóm đã quyết định thực hiện đề tài: "Nghiên cứu mô hình cung cấp tài nguyên trong điện toán đám mây - Case Study xây dựng máy chủ ảo Private với mã nguồn mở OpenStack" nhằm nắm vững công nghệ lõi và quy trình triển khai một hệ thống Cloud tiêu chuẩn công nghiệp.

2. Mục tiêu của đề tài

Về lý thuyết: Nghiên cứu sâu về kiến trúc các thành phần cốt lõi của OpenStack (Nova, Neutron, Keystone, Glance, Cinder, Horizon) và cơ chế cung cấp tài nguyên (Provisioning) trong Cloud.

Về thực hành: Triển khai thành công hệ thống OpenStack phiên bản 2024.1 (Caracal) trên nền tảng hệ điều hành Ubuntu 24.04 LTS.

Về giải pháp: Ứng dụng công cụ Kolla-Ansible để tự động hóa quy trình triển khai, giúp quản lý các dịch vụ OpenStack dưới dạng Docker containers.

Về vận hành: Xây dựng thành công các máy chủ ảo (Instances), cấu hình mạng (VPC, Floating IP) và quản trị lưu trữ để phục vụ nhu cầu thực tế.

3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Hệ sinh thái mã nguồn mở OpenStack phiên bản Caracal.

Cơ chế điều phối và tự động hóa của Ansible trong việc quản trị hạ tầng.

Công nghệ ảo hóa lồng nhau (Nested Virtualization) và Docker Container.

- **Phạm vi nghiên cứu:** Triển khai mô hình Multi-node bao gồm 04 Nodes (Controller, Network, Compute, Storage).

+ Thực hiện trong môi trường ảo hóa KVM/Libvirt trên hệ điều hành Ubuntu 24.04.

+ Tập trung vào các dịch vụ nền tảng (Core Services) để đảm bảo hệ thống Cloud hoạt động ổn định.

4. Phương pháp nghiên cứu

- **Phương pháp nghiên cứu tài liệu:** Thu thập thông tin từ tài liệu chính thống của OpenStack Foundation (docs.openstack.org), các diễn đàn công nghệ và tài liệu kỹ thuật về Kolla-Ansible.

- **Phương pháp phân tích và thiết kế:** Xây dựng sơ đồ logic hệ thống mạng, tính toán thông số phần cứng (CPU, RAM, Storage) phù hợp cho mô hình Lab ảo hóa lồng nhau.

- **Phương pháp thực nghiệm (Case Study):** trực tiếp cài đặt, cấu hình và sửa lỗi (troubleshooting) trên hệ thống thực tế. Áp dụng các bản vá lỗi (hotfix) cho phiên bản Ubuntu 24.04 và Python 3.12 để tối ưu hóa hiệu suất hệ thống.

- **Phương pháp kiểm thử:** Đánh giá kết quả triển khai thông qua việc khởi tạo máy ảo, kiểm tra kết nối mạng và khả năng lưu trữ của hệ thống sau khi "Deploy" thành công.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về Điện toán đám mây (Cloud Computing)

1.1.1. Định nghĩa và đặc điểm

Điện toán đám mây (Cloud Computing) là một mô hình cho phép truy cập mạng thuận tiện, theo yêu cầu, đến một tập hợp tài nguyên điện toán dùng chung có thể cấu hình được, bao gồm mạng, máy chủ, lưu trữ, ứng dụng và các dịch vụ. Các tài nguyên này có thể được cung cấp và thu hồi nhanh chóng với nỗ lực quản lý tối thiểu hoặc tương tác hạn chế từ phía nhà cung cấp dịch vụ. Định nghĩa này nhấn mạnh rằng điện toán đám mây không chỉ là vấn đề lưu trữ dữ liệu trên Internet, mà là một phương thức cung cấp tài nguyên CNTT mang tính cách mạng, làm thay đổi hoàn toàn cách thức xây dựng, triển khai và vận hành hệ thống thông tin.

Một trong những đặc điểm cốt lõi của điện toán đám mây là tính tự phục vụ theo yêu cầu (On-demand Self-service). Đặc điểm này cho phép người dùng chủ động khởi tạo, mở rộng hoặc thu hồi các tài nguyên như máy ảo, dung lượng lưu trữ hay băng thông mạng một cách tự động thông qua giao diện quản lý hoặc API, mà không cần sự can thiệp trực tiếp của nhân sự từ phía nhà cung cấp. Điều này giúp rút ngắn đáng kể thời gian triển khai hệ thống so với mô hình hạ tầng CNTT truyền thống.

Bên cạnh đó, khả năng truy cập mạng rộng khắp (Broad Network Access) đảm bảo rằng các tài nguyên đám mây có thể được truy cập thông qua các giao thức chuẩn và từ nhiều loại thiết bị khác nhau, bao gồm máy tính cá nhân, máy tính bảng, điện thoại thông minh và các thiết bị đầu cuối mỏng (thin client). Nhờ vậy, điện toán đám mây hỗ trợ mạnh mẽ cho các mô hình làm việc linh hoạt như làm việc từ xa (remote working) và làm việc phân tán.

Sự hội tụ tài nguyên (Resource Pooling) là một đặc điểm quan trọng khác, trong đó nhà cung cấp dịch vụ sử dụng mô hình đa người thuê (Multi-tenancy) để phục vụ đồng thời nhiều khách hàng trên cùng một hạ tầng vật lý. Các tài nguyên được phân bổ động theo nhu cầu sử dụng thực tế, giúp tối ưu hóa hiệu suất và giảm chi phí vận hành. Người dùng không cần biết chính xác vị trí vật lý của tài nguyên, mà chỉ quan tâm đến mức dịch vụ được cung cấp.

Ngoài ra, khả năng co giãn nhanh chóng (Rapid Elasticity) cho phép hệ thống tự động mở rộng hoặc thu hẹp tài nguyên gần như tức thời, đáp ứng kịp thời sự biến động của lưu lượng truy cập hoặc khối lượng công việc. Cuối cùng, dịch vụ đo lường được (Measured Service) giúp việc sử dụng tài nguyên được giám sát, ghi nhận và tính phí một cách minh bạch, theo mô hình “trả tiền theo mức sử dụng” (pay-as-you-go). Điều này mang lại lợi ích kinh tế rõ rệt, đặc biệt đối với các doanh nghiệp vừa và nhỏ, khi không cần đầu tư lớn ban đầu cho hạ tầng.

1.1.2. Các mô hình dịch vụ (IaaS, PaaS, SaaS)

Điện toán đám mây được tổ chức theo cấu trúc phân tầng, trong đó mỗi tầng dịch vụ đại diện cho một mức độ trừu tượng hóa khác nhau, nhằm đáp ứng nhu cầu đa dạng của người dùng.

Hạ tầng như một dịch vụ (Infrastructure as a Service – IaaS) là mô hình dịch vụ cơ bản nhất, cung cấp các tài nguyên hạ tầng ảo hóa như máy chủ ảo, hệ thống lưu trữ, mạng và tường lửa. Ở mô hình này, nhà cung cấp chịu trách nhiệm quản lý hạ tầng vật lý, trong khi người dùng có toàn quyền kiểm soát hệ điều hành, middleware, ứng dụng và dữ liệu. IaaS phù hợp với các tổ chức cần sự linh hoạt cao, muốn tự xây dựng và tùy chỉnh hệ thống theo yêu cầu riêng, chẳng hạn như triển khai trung tâm dữ liệu ảo hoặc các môi trường thử nghiệm (testbed).

Nền tảng như một dịch vụ (Platform as a Service – PaaS) nâng mức trừu tượng hóa lên cao hơn bằng cách ẩn đi các chi tiết phức tạp liên quan đến hạ tầng và hệ điều hành. Nhà cung cấp PaaS cung cấp sẵn các nền tảng phát triển, framework, cơ sở dữ liệu và công cụ hỗ trợ, giúp các nhà phát triển tập trung hoàn toàn vào việc xây dựng logic nghiệp vụ của ứng dụng. Mô hình này giúp giảm đáng kể thời gian phát triển phần mềm, đồng thời hạn chế các rủi ro liên quan đến cấu hình sai hạ tầng hoặc thiếu vá lỗi bảo mật.

Ở tầng cao nhất, Phần mềm như một dịch vụ (Software as a Service – SaaS) cung cấp các ứng dụng hoàn chỉnh cho người dùng cuối thông qua trình duyệt web hoặc giao diện nhẹ. Trong mô hình này, toàn bộ quá trình quản lý từ hạ tầng, nền tảng cho đến cập nhật phần mềm đều do nhà cung cấp đảm nhiệm. Người dùng chỉ cần sử dụng dịch vụ mà không cần quan tâm đến các vấn đề kỹ thuật phía sau. SaaS đặc biệt phổ biến trong các ứng dụng văn phòng trực tuyến, quản lý doanh nghiệp và hệ thống quản trị quan hệ khách hàng (CRM), nhờ tính tiện lợi và khả năng triển khai nhanh chóng.

1.1.3. Các mô hình triển khai (Public, Private, Hybrid Cloud)

Bên cạnh các mô hình dịch vụ, điện toán đám mây còn được phân loại theo mô hình triển khai, dựa trên cách thức sở hữu và vận hành hạ tầng.

Public Cloud (Đám mây công cộng) là mô hình trong đó hạ tầng được sở hữu và vận hành bởi các nhà cung cấp dịch vụ lớn, phục vụ cho nhiều khách hàng khác nhau qua Internet. Ưu điểm nổi bật của Public Cloud là khả năng mở rộng gần như vô hạn, chi phí đầu tư ban đầu thấp và thời gian triển khai nhanh. Tuy nhiên, việc chia sẻ hạ tầng cũng đặt ra các thách thức liên quan đến bảo mật, quyền riêng tư và tuân thủ quy định pháp lý.

Ngược lại, Private Cloud (Đám mây dùng riêng) – trọng tâm nghiên cứu của đề tài – được xây dựng dành riêng cho một tổ chức hoặc doanh nghiệp duy nhất. Hạ tầng có thể được triển khai tại trung tâm dữ liệu nội bộ hoặc do bên thứ ba quản lý, nhưng vẫn đảm bảo quyền kiểm soát tuyệt đối của doanh nghiệp đối với dữ liệu và hệ thống. Private Cloud cho phép tùy chỉnh sâu về bảo mật, hiệu năng và kiến trúc, đặc biệt phù hợp với các tổ chức tài chính, y tế hoặc cơ quan nhà nước, nơi yêu cầu nghiêm ngặt về an toàn thông tin và tuân thủ pháp lý.

Hybrid Cloud (Đám mây lai) là sự kết hợp giữa Public Cloud và Private Cloud, cho phép doanh nghiệp khai thác ưu điểm của cả hai mô hình. Các ứng dụng và dữ liệu nhạy cảm có thể được lưu trữ và xử lý trên Private Cloud, trong khi các tác vụ ít nhạy cảm hoặc cần khả năng mở rộng lớn được triển khai trên Public Cloud. Xu hướng Hybrid Cloud ngày càng phổ biến trong bối cảnh các quy định về chủ quyền dữ liệu (ví dụ như GDPR) ngày càng chặt chẽ, đồng thời

doanh nghiệp vẫn cần tận dụng tính linh hoạt và khả năng mở rộng của đám mây công cộng.

Tóm lại, điện toán đám mây, đặc biệt là mô hình Private Cloud, mang lại một nền tảng linh hoạt, an toàn và hiệu quả cho việc xây dựng hệ thống CNTT hiện đại. Việc hiểu rõ các đặc điểm, mô hình dịch vụ và mô hình triển khai là cơ sở quan trọng để thiết kế và triển khai các giải pháp đám mây phù hợp với mục tiêu và điều kiện cụ thể của từng tổ chức.

1.2. Giới thiệu về mã nguồn mở OpenStack

1.2.1. Lịch sử hình thành và phiên bản 2024.1 (Caracal)

OpenStack là một nền tảng điện toán đám mây mã nguồn mở được thiết kế nhằm xây dựng và vận hành các hệ thống Cloud Computing, đặc biệt là Private Cloud và Hybrid Cloud. OpenStack không chỉ đơn thuần là một sản phẩm phần mềm, mà còn là kết quả của một phong trào công nghệ toàn cầu, nơi các tổ chức, doanh nghiệp và cộng đồng cùng đóng góp để xây dựng một chuẩn mở cho hạ tầng đám mây.

Dự án OpenStack chính thức ra đời vào năm 2010, bắt nguồn từ sự kết hợp giữa dự án Nebula của NASA (phục vụ quản lý tài nguyên tính toán cho các nhiệm vụ nghiên cứu) và hệ thống lưu trữ đám mây của Rackspace. Mục tiêu ban đầu của OpenStack là tạo ra một nền tảng đám mây có khả năng mở rộng lớn, không phụ thuộc vào nhà cung cấp (vendor-neutral), cho phép các tổ chức tự xây dựng hạ tầng tương tự các nhà cung cấp đám mây công cộng lớn.

Trong suốt hơn 14 năm phát triển, OpenStack đã trải qua gần 30 phiên bản phát hành chính thức, mỗi phiên bản được đặt tên theo thứ tự bảng chữ cái và một địa danh hoặc sinh vật, thể hiện sự tiến hóa liên tục của hệ sinh thái. Từ các phiên bản đầu tiên còn mang tính thử nghiệm, OpenStack đã dần trở thành nền tảng Cloud trưởng thành, được triển khai rộng rãi trong các trung tâm dữ liệu của doanh nghiệp, tổ chức chính phủ và nhà cung cấp dịch vụ viễn thông.

Phiên bản 2024.1 (Caracal) là một cột mốc quan trọng trong lộ trình phát triển theo mô hình SLURP (Skip-Level Upgrade Release Process), trong đó mỗi năm OpenStack phát hành một phiên bản chính hỗ trợ dài hạn, giúp các tổ chức dễ dàng nâng cấp và duy trì hệ thống ổn định. Caracal tập trung mạnh vào việc hiện đại hóa mã nguồn, loại bỏ dần các thành phần lỗi thời và tối ưu hóa hiệu suất cho các workload hiện đại.

Một trong những điểm nổi bật của Caracal là việc hỗ trợ đầy đủ Python 3.12, đảm bảo tính tương thích với các thư viện và công cụ mới nhất, đồng thời nâng cao hiệu năng xử lý. Ngoài ra, phiên bản này tăng cường khả năng tích hợp với các công nghệ tăng tốc phần cứng như GPU, SmartNIC và DPU, vốn đóng vai trò then chốt trong các ứng dụng trí tuệ nhân tạo (AI), học sâu (Deep Learning) và xử lý dữ liệu lớn (Big Data).

Bên cạnh đó, OpenStack Caracal cũng cải thiện đáng kể cơ chế bảo mật, đặc biệt là RBAC (Role-Based Access Control), cho phép quản trị viên định nghĩa vai trò và quyền hạn một cách chi tiết hơn trong môi trường đa người dùng, đa dự án. Giao diện quản trị và trải nghiệm người dùng cũng được tinh chỉnh nhằm giảm độ phức tạp trong quá trình vận hành và giám sát hệ thống.

1.2.2. Các thành phần cốt lõi

Kiến trúc của OpenStack được xây dựng theo mô hình hướng dịch vụ (Service-Oriented Architecture – SOA), trong đó mỗi chức năng chính của hệ thống được đảm nhiệm bởi một dịch vụ (service) độc lập, giao tiếp với nhau thông qua API chuẩn RESTful. Cách tiếp cận này giúp hệ thống có khả năng mở rộng linh hoạt, dễ bảo trì và dễ tích hợp với các công nghệ bên ngoài.

Các thành phần cốt lõi của OpenStack bao gồm:

Keystone (Identity Service): Keystone đóng vai trò là trung tâm xác thực và quản lý danh tính của toàn bộ hệ thống OpenStack. Dịch vụ này chịu trách nhiệm xác thực người dùng, cấp phát Token truy cập, quản lý dự án (project), vai trò (role) và danh mục dịch vụ (service catalog). Mọi yêu cầu đến các dịch vụ khác như Nova hay Neutron đều phải thông qua Keystone để kiểm tra quyền hạn, đảm bảo tính bảo mật và kiểm soát truy cập chặt chẽ.

Nova (Compute Service): Nova là dịch vụ trung tâm quản lý tài nguyên tính toán, chịu trách nhiệm khởi tạo, dừng, hủy và giám sát các máy ảo (instances). Nova bao gồm nhiều thành phần con như Nova API, Nova Scheduler và Nova Compute, phối hợp với nhau để quyết định máy ảo sẽ được triển khai trên Node nào, dựa trên tình trạng tài nguyên và chính sách phân bổ.

Neutron (Networking Service): Neutron cung cấp khả năng mạng như một dịch vụ (Network as a Service – NaaS), cho phép tạo và quản lý các mạng ảo, subnet, router, địa chỉ IP động và IP tĩnh (Floating IP). Ngoài ra, Neutron còn hỗ trợ các tính năng bảo mật mạng như Security Groups và Network Policies, giúp cô lập và bảo vệ các máy ảo trong môi trường đa tenant.

Glance (Image Service): Glance là kho lưu trữ và quản lý các image hệ điều hành, chẳng hạn như file ISO hoặc QCOW2. Các image này được Nova sử dụng để tạo ra các máy ảo mới. Việc tách riêng Glance giúp chuẩn hóa và đơn giản hóa quá trình quản lý image trong toàn hệ thống.

Cinder (Block Storage Service): Cinder cung cấp dịch vụ lưu trữ khối (block storage), cho phép tạo các ổ đĩa ảo có thể gắn hoặc tháo rời linh hoạt vào máy ảo. Cinder hỗ trợ nhiều backend lưu trữ khác nhau, từ LVM truyền thống đến các hệ thống lưu trữ phân tán như Ceph, giúp nâng cao tính sẵn sàng và khả năng mở rộng của dữ liệu.

Horizon (Dashboard): Horizon là giao diện web trực quan cho phép người dùng và quản trị viên thao tác với OpenStack thông qua trình duyệt. Horizon giúp giảm sự phụ thuộc vào dòng lệnh, cung cấp cái nhìn tổng quan về tài nguyên và trạng thái hệ thống, từ đó nâng cao hiệu quả quản trị.

1.2.3. Cơ chế hoạt động và tương tác giữa các Service

Cơ chế vận hành của OpenStack dựa trên mô hình giao tiếp phân tán và trao đổi thông điệp không đồng bộ giữa các dịch vụ. Khi người dùng gửi yêu cầu tạo máy ảo thông qua Horizon hoặc API, yêu cầu này trước hết được chuyển đến Keystone để xác thực danh tính và kiểm tra quyền truy cập.

Sau khi được xác thực, yêu cầu sẽ được Nova API tiếp nhận và chuyển cho Nova Scheduler. Scheduler có nhiệm vụ phân tích trạng thái tài nguyên của các Compute Node, bao gồm CPU, RAM và dung lượng lưu trữ, để lựa chọn node phù hợp nhất theo các thuật toán lập lịch và chính sách đã cấu hình. Khi node đích được xác định, Nova Compute sẽ thực hiện quá trình khởi tạo máy ảo.

Trong quá trình này, Nova sẽ gọi Glance để lấy image hệ điều hành tương ứng, đồng thời tương tác với Neutron để cấp phát mạng, gán địa chỉ IP và cấu hình các quy tắc bảo mật. Nếu người dùng yêu cầu thêm lưu trữ, Cinder sẽ tạo volume và ánh xạ vào máy ảo thông qua các giao thức như iSCSI hoặc các cơ chế tương đương.

Toàn bộ thông tin trạng thái, cấu hình và metadata của hệ thống được lưu trữ trong cơ sở dữ liệu trung tâm như MariaDB. Việc lưu trữ tập trung này đảm bảo rằng ngay cả khi một dịch vụ bị khởi động lại hoặc gặp sự cố, hệ thống vẫn có thể khôi phục và duy trì tính nhất quán của hạ tầng.

Nhờ kiến trúc phân tán, hướng dịch vụ và khả năng mở rộng linh hoạt, OpenStack trở thành một nền tảng lý tưởng để xây dựng Private Cloud quy mô lớn, đáp ứng các yêu cầu khắt khe về bảo mật, hiệu năng và khả năng tùy biến trong môi trường doanh nghiệp hiện đại.

1.3. Công nghệ Container và Tự động hóa trong OpenStack

Sự phát triển của OpenStack kéo theo độ phức tạp ngày càng cao trong việc triển khai, vận hành và bảo trì hệ thống. Một hệ thống OpenStack hoàn chỉnh có thể bao gồm hàng chục dịch vụ khác nhau, mỗi dịch vụ lại phụ thuộc vào nhiều thư viện, phiên bản ngôn ngữ và cấu hình riêng biệt. Trong bối cảnh đó, các công nghệ Container hóa và Tự động hóa hạ tầng đóng vai trò then chốt, giúp OpenStack chuyển mình từ một nền tảng khó triển khai thành một giải pháp Cloud sẵn sàng cho môi trường doanh nghiệp.

1.3.1. Docker và Containerization

Trong giai đoạn đầu, OpenStack thường được triển khai theo phương pháp cài đặt trực tiếp lên hệ điều hành (Native Install). Cách tiếp cận này nhanh chóng bộc lộ nhiều hạn chế, đặc biệt là hiện tượng được cộng đồng gọi là “Dependency Hell” – nơi các thư viện Python, phiên bản OpenSSL hay các gói hệ thống của dịch vụ này xung đột với dịch vụ khác. Khi hệ thống mở rộng, việc cập nhật hoặc vá lỗi cho một thành phần có thể gây ảnh hưởng dây chuyền đến toàn bộ nền tảng.

Sự ra đời của Docker đã tạo ra một bước ngoặt quan trọng trong cách triển khai OpenStack. Docker giới thiệu khái niệm Containerization, cho phép đóng gói từng dịch vụ OpenStack (ví dụ: nova-api, neutron-server, keystone) cùng toàn bộ môi trường thực thi của nó, bao gồm thư viện, biến môi trường và tệp cấu

hình, vào các container độc lập. Nhờ đó, mỗi dịch vụ hoạt động như một “hộp đen” biệt lập, không còn phụ thuộc trực tiếp vào môi trường hệ điều hành bên dưới.

Việc sử dụng container mang lại nhiều lợi ích rõ rệt. Trước hết, tính nhất quán môi trường được đảm bảo tuyệt đối: một container chạy ổn định trong môi trường thử nghiệm (Lab) sẽ có hành vi tương tự khi được triển khai lên môi trường sản xuất (Production). Thứ hai, khả năng cô lập lỗi được nâng cao; nếu một dịch vụ gặp sự cố hoặc treo tiến trình, quản trị viên chỉ cần khởi động lại container tương ứng trong vài giây mà không làm gián đoạn toàn bộ máy chủ.

Ngoài ra, container còn giúp tối ưu việc sử dụng tài nguyên hệ thống so với máy ảo truyền thống, do container chia sẻ kernel của hệ điều hành chủ. Điều này đặc biệt quan trọng trong các cụm OpenStack lớn, nơi hiệu suất và mật độ dịch vụ là yếu tố then chốt. Nhờ Containerization, OpenStack từng bước tiếp cận các mô hình vận hành hiện đại tương tự các nền tảng Cloud-native.

1.3.2. Ansible – Công cụ quản lý cấu hình

Bên cạnh vấn đề đóng gói dịch vụ, bài toán triển khai và cấu hình OpenStack trên quy mô lớn cũng đặt ra nhiều thách thức. Một hệ thống Cloud thực tế có thể bao gồm hàng chục, thậm chí hàng trăm máy chủ với vai trò khác nhau như Controller, Compute, Storage và Network Node. Việc cấu hình thủ công từng máy chủ, chỉnh sửa hàng trăm tệp .conf không chỉ tốn thời gian mà còn tiềm ẩn rủi ro sai sót do yếu tố con người.

Ansible xuất hiện như một giải pháp tự động hóa hiệu quả, dựa trên triết lý Infrastructure as Code (IaC). Thay vì cấu hình hệ thống bằng thao tác thủ công, toàn bộ trạng thái mong muốn của hạ tầng được mô tả dưới dạng các Playbook viết bằng ngôn ngữ YAML, có cú pháp đơn giản và dễ đọc. Quản trị viên chỉ cần định nghĩa hệ thống cần ở trạng thái nào, Ansible sẽ tự động thực hiện các bước cần thiết để đạt được trạng thái đó.

Một ưu điểm nổi bật của Ansible là cơ chế agentless, sử dụng giao thức SSH để kết nối và triển khai cấu hình, không yêu cầu cài đặt phần mềm bổ sung trên các Node đích. Điều này giúp giảm độ phức tạp trong vận hành và tăng tính bảo mật. Hơn nữa, Ansible đảm bảo tính idempotency, nghĩa là một Playbook có thể được chạy nhiều lần mà không làm thay đổi kết quả cuối cùng nếu hệ thống đã ở đúng trạng thái mong muốn.

Trong bối cảnh OpenStack, Ansible cho phép tự động hóa toàn bộ quy trình từ cài đặt gói phần mềm, cấu hình dịch vụ, khởi động container cho đến kiểm tra trạng thái hệ thống. Nhờ đó, thời gian triển khai được rút ngắn đáng kể, đồng thời đảm bảo tính lặp lại và nhất quán – những yếu tố sống còn trong môi trường Cloud doanh nghiệp.

1.3.3. Kolla-Ansible: Ưu điểm so với các phương pháp cài đặt truyền thống

Kolla-Ansible là dự án chính thức thuộc hệ sinh thái OpenStack, được phát triển nhằm kết hợp sức mạnh của Docker và Ansible để cung cấp một giải pháp triển khai OpenStack hoàn chỉnh, ổn định và sẵn sàng cho môi trường sản

xuất. Kolla-Ansible cung cấp sẵn các Docker Image chuẩn hóa cho từng dịch vụ OpenStack, cùng với các Ansible Playbook được thiết kế theo các best practice của cộng đồng.

So với các phương pháp cài đặt truyền thống, điểm vượt trội nhất của Kolla-Ansible nằm ở quản lý vòng đời hệ thống (Lifecycle Management). Các tác vụ như triển khai mới, mở rộng (scale-out), nâng cấp phiên bản hay thay đổi cấu hình đều có thể được thực hiện thông qua các lệnh tự động hóa, thay vì phải can thiệp thủ công vào từng máy chủ. Đặc biệt, cơ chế Rolling Upgrade cho phép nâng cấp OpenStack từ phiên bản cũ lên các phiên bản mới như Caracal mà không gây gián đoạn toàn bộ hệ thống, đảm bảo tính sẵn sàng cao cho dịch vụ.

Ngoài ra, Kolla-Ansible tích hợp hệ thống Prechecks rất chặt chẽ, giúp kiểm tra trước các điều kiện cần thiết như cấu hình mạng, dung lượng đĩa, bộ nhớ và quyền truy cập. Nhờ đó, các lỗi tiềm ẩn có thể được phát hiện và xử lý sớm, tránh những sự cố nghiêm trọng trong quá trình cài đặt hoặc nâng cấp.

Đối với doanh nghiệp, việc sử dụng Kolla-Ansible mang lại lợi ích kinh tế rõ rệt. Chi phí vận hành (OPEX) được giảm đáng kể do đội ngũ kỹ thuật không còn phải dành nhiều thời gian cho việc khắc phục lỗi triển khai hay xử lý xung đột phụ thuộc. Thay vào đó, họ có thể tập trung vào việc khai thác, tối ưu và phát triển các dịch vụ chạy trên nền tảng OpenStack, qua đó nâng cao giá trị sử dụng của hạ tầng Cloud.

CHƯƠNG 2: THIẾT KẾ VÀ XÂY DỰNG

2.1. Phân tích yêu cầu hệ thống

Để xây dựng một hệ thống Private Cloud hoạt động ổn định và hiệu quả trên nền tảng OpenStack Caracal (phiên bản 2024.1), việc phân tích và xác định rõ ràng các yêu cầu về phần cứng, hệ điều hành và phần mềm là bước mang tính quyết định. Quá trình này không chỉ đảm bảo hệ thống vận hành đúng chức năng, mà còn ảnh hưởng trực tiếp đến hiệu năng, khả năng mở rộng và độ ổn định lâu dài của toàn bộ hạ tầng Cloud.

Trong phạm vi đề tài, hệ thống được triển khai theo mô hình ảo hóa lồng nhau (Nested Virtualization), trong đó một máy vật lý (Host) đóng vai trò nền tảng để khởi tạo và vận hành nhiều máy ảo (Virtual Machines – VMs). Trên các máy ảo này, OpenStack tiếp tục triển khai các máy ảo con (Instance) phục vụ người dùng cuối. Mô hình này thường được áp dụng trong môi trường nghiên cứu, học tập hoặc thử nghiệm (Lab), nơi việc đầu tư nhiều máy chủ vật lý riêng biệt là không khả thi.

Máy Host là thành phần quan trọng nhất của hệ thống, chịu trách nhiệm cung cấp toàn bộ tài nguyên cho các máy ảo OpenStack. Do phải gánh nhiều lớp ảo hóa, máy Host cần có cấu hình đủ mạnh để tránh hiện tượng nghẽn tài nguyên.

- Bộ nhớ RAM: Dung lượng RAM tối thiểu được đề xuất là 16GB, trong đó mỗi máy ảo OpenStack thành phần (Controller, Compute, Network/Storage, Monitoring) cần tối thiểu từ 3–4GB RAM để hoạt động ổn định. Tuy nhiên, để đảm bảo dư địa cho hệ điều hành Host, Docker Container và các tiến trình nền, cấu hình 32GB RAM được khuyến nghị nhằm nâng cao hiệu suất và giảm nguy cơ xảy ra tình trạng swap bộ nhớ.

- Bộ xử lý (CPU): CPU của máy Host phải hỗ trợ công nghệ ảo hóa phần cứng như Intel VT-x hoặc AMD-V, cho phép kích hoạt Nested Virtualization. Đây là điều kiện bắt buộc để các máy ảo OpenStack có thể sử dụng trình ảo hóa KVM bên trong máy ảo. Ngoài ra, CPU đa nhân (tối thiểu 8 luồng xử lý) sẽ giúp phân bổ tài nguyên linh hoạt hơn cho các VM và Instance được tạo trong OpenStack.

- Lưu trữ: Hệ thống cần ổ cứng SSD với dung lượng tối thiểu 200GB để đáp ứng nhu cầu lưu trữ image hệ điều hành, container Docker, volume Cinder và log hệ thống. Việc sử dụng SSD thay vì HDD truyền thống giúp cải thiện đáng kể tốc độ khởi tạo máy ảo và thời gian phản hồi của dịch vụ.

Hệ điều hành được lựa chọn cho cả máy Host và các máy ảo OpenStack là Ubuntu 24.04 LTS. Đây là phiên bản hỗ trợ dài hạn (Long Term Support), mang lại nhiều lợi thế cho hệ thống Private Cloud:

- Tính ổn định và bảo mật lâu dài, với chu kỳ hỗ trợ lên đến 5 năm, phù hợp cho các hệ thống cần vận hành liên tục.

- Khả năng tương thích cao với các phiên bản mới của Linux Kernel, giúp tối ưu hóa hiệu suất cho các công nghệ nền tảng như Docker, KVM và Open vSwitch.

- Hệ sinh thái phần mềm phong phú, được cộng đồng OpenStack và Docker hỗ trợ chính thức, giảm thiểu rủi ro xung đột phiên bản trong quá trình triển khai.

- Việc đồng nhất hệ điều hành giữa Host và các VM cũng giúp đơn giản hóa quá trình quản trị, giám sát và khắc phục sự cố.

Để triển khai OpenStack Caracal theo hướng hiện đại và ổn định, hệ thống cần các thành phần phần mềm sau:

- Docker: Docker đóng vai trò nền tảng cho việc container hóa các dịch vụ OpenStack. Mỗi dịch vụ như Nova, Neutron, Keystone được đóng gói trong các container riêng biệt, giúp giảm xung đột phụ thuộc và tăng khả năng cô lập lỗi.

- Kolla-Ansible: Kolla-Ansible phiên bản 19.x được sử dụng nhằm đảm bảo tương thích đầy đủ với OpenStack Caracal. Kolla-Ansible cung cấp sẵn các Docker Image và Ansible Playbook chuẩn hóa, cho phép triển khai OpenStack nhanh chóng, nhất quán và dễ dàng mở rộng.

- Ansible: Là công cụ tự động hóa hạ tầng theo mô hình Infrastructure as Code, Ansible giúp quản lý cấu hình và triển khai OpenStack trên nhiều Node một cách đồng bộ, hạn chế tối đa lỗi do thao tác thủ công.

2.2. Mô hình kiến trúc triển khai

Trong triển khai hệ thống Private Cloud quy mô trung bình và lớn, mô hình Multi-node Architecture được xem là kiến trúc tiêu chuẩn nhằm đảm bảo tính linh hoạt, khả năng mở rộng và mức độ an toàn cao. Thay vì gom toàn bộ dịch vụ vào một máy chủ duy nhất (All-in-One), mô hình Multi-node phân tách rõ ràng các vai trò chức năng của từng Node, từ đó giảm tải cho từng thành phần và hạn chế ảnh hưởng dây chuyền khi xảy ra sự cố.

Kiến trúc này cũng phản ánh sát với các mô hình triển khai OpenStack trong môi trường doanh nghiệp và trung tâm dữ liệu thực tế, do đó rất phù hợp với mục tiêu nghiên cứu và thực nghiệm của đề tài.

2.2.1. Sơ đồ logic hệ thống

Hệ thống được thiết kế theo mô hình Multi-node với việc tách biệt các chức năng quản trị, mạng, tính toán và lưu trữ thành các Node riêng biệt. Mỗi Node đảm nhiệm một vai trò cụ thể trong hệ sinh thái OpenStack, giúp tối ưu hiệu năng và tăng cường bảo mật.

Cụ thể, hệ thống bao gồm các thành phần sau:

- Controller Node: Controller Node đóng vai trò là “đầu não” của toàn bộ hệ thống OpenStack. Node này chịu trách nhiệm vận hành các dịch vụ quản lý trung tâm như Keystone (Identity), Nova API (Compute Management), Neutron Server (Network Management), Glance (Image

Service) và Horizon (Dashboard). Ngoài ra, Controller Node còn lưu trữ cơ sở dữ liệu và hệ thống message queue, đảm bảo sự đồng bộ và nhất quán trạng thái giữa các dịch vụ. Do tính chất quan trọng này, Controller Node yêu cầu mức độ ổn định và bảo mật cao nhất trong toàn hệ thống.

- Network Node: Network Node chuyên trách xử lý toàn bộ lưu lượng mạng của hệ thống, bao gồm định tuyến (routing), NAT, DHCP và kết nối giữa mạng nội bộ và mạng bên ngoài. Node này triển khai các thành phần mạng cốt lõi của OpenStack Neutron, thường kết hợp với các công nghệ ảo hóa mạng như Open vSwitch. Việc tách riêng Network Node giúp tránh tình trạng nghẽn cổ chai khi lưu lượng mạng tăng cao, đồng thời tăng cường khả năng kiểm soát và giám sát an ninh mạng.

- Compute Node: Compute Node là nơi trực tiếp thực thi các máy ảo của người dùng (Instances). Node này chạy dịch vụ Nova Compute và sử dụng trình ảo hóa KVM để khởi tạo, quản lý vòng đời và giám sát trạng thái các máy ảo. Tài nguyên CPU và RAM của Compute Node được phân bổ linh hoạt cho từng Instance, cho phép hệ thống mở rộng theo chiều ngang bằng cách bổ sung thêm Compute Node khi nhu cầu tăng cao.

- Storage Node: Storage Node cung cấp tài nguyên lưu trữ tập trung cho toàn hệ thống, phục vụ cho lưu trữ block (Cinder) hoặc image hệ điều hành. Trong các mô hình nâng cao, Node này có thể tích hợp các hệ thống lưu trữ phân tán như Ceph để tăng tính sẵn sàng và khả năng chịu lỗi. Việc tách Storage Node giúp đảm bảo dữ liệu không phụ thuộc trực tiếp vào Compute Node, hạn chế rủi ro mất dữ liệu khi Compute Node gặp sự cố.

Nhờ việc phân tách rõ ràng vai trò của từng Node, kiến trúc Multi-node giúp hệ thống đạt được sự cân bằng giữa hiệu năng – khả năng mở rộng – mức độ an toàn, đồng thời tạo tiền đề thuận lợi cho các bước mở rộng và nâng cấp trong tương lai.

2.2.2. Quy hoạch địa chỉ IP và Network Topology

Mạng đóng vai trò xương sống trong hệ thống OpenStack Multi-node. Do đó, việc quy hoạch địa chỉ IP và thiết kế Network Topology cần được thực hiện cẩn thận ngay từ đầu để tránh xung đột, đảm bảo khả năng quản trị và mở rộng lâu dài.

Trong phạm vi đề tài, hệ thống mạng được phân tách thành hai dải mạng chính, mỗi dải đảm nhiệm một chức năng riêng biệt:

- Management Network (Mạng quản trị): Mạng quản trị sử dụng dải địa chỉ 10.0.0.0/24, đóng vai trò là kênh giao tiếp nội bộ giữa các Node trong hệ thống. Trên mạng này, các dịch vụ API của OpenStack, message queue, cơ sở dữ liệu và các tác vụ quản trị được truyền tải. Việc cô lập Management Network giúp tăng cường bảo mật, hạn chế nguy cơ tấn công từ bên ngoài và đảm bảo độ ổn định cho các dịch vụ điều khiển quan trọng.

- Provider / External Network (Mạng cung cấp dịch vụ): Mạng Provider hay External Network được sử dụng để cấp phát địa chỉ IP cho

các máy ảo người dùng (Instances), cho phép chúng truy cập ra Internet hoặc kết nối với hệ thống bên ngoài. Mạng này thường được ánh xạ trực tiếp với card mạng vật lý của Network Node thông qua cơ chế bridge hoặc VLAN, đảm bảo hiệu năng cao và giảm độ trễ khi truyền dữ liệu.

2.2.3. Đánh giá mô hình kiến trúc Multi-node

Mô hình Multi-node Architecture mang lại nhiều ưu điểm vượt trội so với mô hình All-in-One, đặc biệt trong các hệ thống Private Cloud hướng đến tính ổn định và khả năng mở rộng. Việc phân tách chức năng theo từng Node giúp hệ thống dễ dàng nâng cấp, bảo trì và xử lý sự cố cục bộ mà không làm gián đoạn toàn bộ dịch vụ.

Trong khuôn khổ đề tài, mô hình này không chỉ đáp ứng tốt yêu cầu thực nghiệm với OpenStack Caracal, mà còn phản ánh sát với kiến trúc triển khai thực tế, tạo nền tảng vững chắc cho các nội dung tiếp theo như triển khai chi tiết từng Node, kiểm thử hiệu năng và đánh giá kết quả hệ thống.

2.3. Chuẩn bị hạ tầng ảo hóa

Trong kiến trúc triển khai OpenStack theo mô hình Multi-node trên môi trường thử nghiệm, ảo hóa lồng nhau (Nested Virtualization) đóng vai trò nền tảng, cho phép mô phỏng một hệ thống Private Cloud hoàn chỉnh chỉ với một máy vật lý duy nhất. Máy Host vật lý đảm nhiệm vai trò Hypervisor tầng 1, trong khi các Node OpenStack (Controller, Network, Compute, Storage) được triển khai dưới dạng máy ảo, tiếp tục cung cấp khả năng ảo hóa cho các Instance người dùng.

Cách tiếp cận này không chỉ tối ưu chi phí phần cứng mà còn phản ánh đúng logic triển khai OpenStack trong thực tế, đặc biệt phù hợp cho mục đích nghiên cứu, đào tạo và kiểm thử.

2.3.1. Cấu hình Máy Host

Máy Host là nền tảng cốt lõi của toàn bộ hệ thống, chịu trách nhiệm cung cấp tài nguyên CPU, RAM và lưu trữ cho tất cả các máy ảo thành phần. Hệ điều hành được lựa chọn là Ubuntu 24.04 LTS nhờ tính ổn định cao, khả năng hỗ trợ dài hạn và mức độ tương thích tốt với các công nghệ ảo hóa hiện đại.

Sau khi hoàn tất cài đặt hệ điều hành, bước đầu tiên là cập nhật toàn bộ hệ thống nhằm đảm bảo các gói phần mềm ở trạng thái mới và ổn định nhất. Tiếp theo, các công cụ ảo hóa cốt lõi được cài đặt bao gồm:

- KVM (Kernel-based Virtual Machine): Là trình ảo hóa cấp kernel, cho phép Linux hoạt động như một hypervisor hiệu năng cao, tận dụng trực tiếp các tính năng ảo hóa phần cứng của CPU.
- Libvirt: Cung cấp lớp API và công cụ quản lý thống nhất cho KVM, cho phép quản trị viên điều khiển máy ảo thông qua dòng lệnh hoặc giao diện đồ họa.
- virt-manager: Là công cụ giao diện đồ họa giúp theo dõi, kiểm soát trạng thái máy ảo và hỗ trợ debug trong giai đoạn triển khai ban đầu.

Một bước cấu hình quan trọng trong giai đoạn này là phân quyền người dùng, cụ thể là thêm User hiện tại vào các nhóm libvirt và kvm. Việc này cho phép người dùng quản lý máy ảo mà không cần sử dụng quyền root liên tục, từ đó:

- Giảm nguy cơ thao tác sai gây ảnh hưởng toàn hệ thống.
- Nâng cao mức độ an toàn và tuân thủ các nguyên tắc bảo mật.

Ngoài ra, Nested Virtualization được kích hoạt ở cấp CPU để đảm bảo các máy ảo Compute Node có thể tiếp tục khởi tạo các Instance bên trong OpenStack.

2.3.2. Thiết lập KVM và mạng ảo

Sau khi hoàn tất cấu hình hypervisor, bước tiếp theo là thiết lập mạng ảo nhằm đảm bảo các máy ảo OpenStack có thể giao tiếp ổn định với nhau và với máy Host. Trong hệ thống này, một Virtual Bridge tên là virbr1 được tạo ra thông qua cấu hình XML của Libvirt.

Mạng ảo này, được đặt tên logic là openstack-net, đảm nhiệm các chức năng sau:

- Đóng vai trò bridge Layer 2, cho phép các Node giao tiếp trực tiếp như trong cùng một mạng vật lý
- Cung cấp dịch vụ DHCP tĩnh (Static DHCP) dựa trên MAC Address của từng máy ảo

Cách tiếp cận DHCP tĩnh mang lại nhiều lợi ích quan trọng cho quá trình triển khai OpenStack:

- Mỗi Node (Controller, Network, Compute, Storage) luôn nhận đúng địa chỉ IP đã quy hoạch
- Tránh tình trạng thay đổi IP sau khi reboot, vốn có thể gây lỗi nghiêm trọng cho các dịch vụ OpenStack
- Đảm bảo tính ổn định và nhất quán khi triển khai tự động bằng Kolla-Ansible

Việc tách riêng mạng ảo nội bộ cho OpenStack cũng giúp cô lập lưu lượng quản trị, giảm nhiễu và thuận lợi cho việc debug khi xảy ra sự cố.

2.4. Xây dựng các Node thành phần

Sau khi hoàn tất hạ tầng ảo hóa, các Node của hệ thống OpenStack được khởi tạo dưới dạng máy ảo bằng công cụ dòng lệnh virt-install. Việc lựa chọn triển khai bằng dòng lệnh thay cho giao diện đồ họa giúp đảm bảo tính đồng nhất cấu hình giữa các Node, đồng thời cho phép tái tạo nhanh chóng toàn bộ môi trường khi cần triển khai lại. Cách tiếp cận này cũng phù hợp với triết lý tự động hóa hạ tầng (Infrastructure as Code), vốn là nền tảng trong các hệ thống Cloud hiện đại.

Controller Node đóng vai trò trung tâm điều phối toàn bộ hệ thống OpenStack, vì vậy được ưu tiên phân bổ tài nguyên lớn hơn so với các Node còn lại. Node này được cấp phát 8GB RAM nhằm đảm bảo khả năng vận hành ổn

định các dịch vụ cốt lõi như MariaDB để lưu trữ trạng thái và metadata của hệ thống, RabbitMQ để xử lý hàng đợi thông điệp giữa các dịch vụ, và Horizon cung cấp giao diện quản trị web cho người dùng. Trong thực tế triển khai, nếu Controller Node không được cấp đủ tài nguyên, hệ thống có thể gặp tình trạng phản hồi API chậm hoặc gián đoạn dịch vụ, ảnh hưởng trực tiếp đến toàn bộ Cloud.

Network Node được thiết kế với hai card mạng ảo nhằm tách biệt rõ ràng các luồng lưu thông dữ liệu. Một card mạng kết nối vào Management Network, đảm nhiệm việc giao tiếp nội bộ giữa các dịch vụ OpenStack, trong khi card còn lại kết nối ra Provider/External Network để thực hiện các chức năng routing, NAT và cho phép các Instance truy cập Internet. Mô hình này phản ánh đúng kiến trúc triển khai OpenStack trong môi trường doanh nghiệp, đồng thời giúp tăng cường khả năng kiểm soát và bảo mật mạng.

Compute Node là thành phần then chốt trong mô hình ảo hóa lồng nhau (Nested Virtualization). Trên Node này, tùy chọn CPU host-passthrough được kích hoạt để máy ảo có thể nhận diện đầy đủ các tập lệnh ảo hóa từ CPU vật lý. Cấu hình này cho phép dịch vụ Nova Compute khởi tạo các Instance con một cách ổn định, tránh các lỗi phổ biến như “CPU does not support virtualization” thường gặp trong môi trường nested, đồng thời cải thiện hiệu năng xử lý cho các máy ảo người dùng. Nếu thiếu cấu hình host-passthrough, Compute Node sẽ không thể thực hiện đúng chức năng tính toán trong OpenStack.

Storage Node được xây dựng nhằm cung cấp tài nguyên lưu trữ block cho các máy ảo OpenStack. Bên cạnh ổ đĩa hệ điều hành, Node này được gắn thêm một ổ đĩa ảo /dev/vdb dung lượng 30GB và được cấu hình bằng LVM để tạo Volume Group mang tên cinder-volumes. Không gian lưu trữ này phục vụ cho dịch vụ Cinder Block Storage, cho phép cấp phát và quản lý volume một cách linh hoạt. Việc tách riêng ổ đĩa lưu trữ khỏi hệ điều hành giúp đơn giản hóa quá trình mở rộng dung lượng, giảm rủi ro ảnh hưởng đến OS và phản ánh đúng kiến trúc lưu trữ trong các hệ thống OpenStack triển khai thực tế.

CHƯƠNG 3: TRIỂN KHAI VÀ THỬ NGHIỆM

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Dựa trên quá trình triển khai thực tế mô hình Multi-node bằng Kolla-Ansible, hệ thống đạt được các kết quả sau:

- Về tính ổn định: Hệ thống hoạt động ổn định trên nền tảng ảo hóa lồng nhau (Nested Virtualization). Các dịch vụ cốt lõi (Nova, Neutron, Keystone, Horizon) được đóng gói trong Docker Container, giúp việc quản lý và cô lập tài nguyên hiệu quả, giảm thiểu xung đột thư viện hệ thống. Cơ chế Self-healing của Docker giúp tự động khởi động lại các container dịch vụ nếu xảy ra sự cố phần mềm đơn lẻ.

- Về tốc độ triển khai: Sử dụng Kolla-Ansible giúp rút ngắn thời gian triển khai từ nhiều ngày (nếu cài thủ công) xuống còn khoảng 45-60 phút. Khả năng tái sử dụng cấu hình (file globals.yml và multinode) cho phép mở rộng quy mô hệ thống (Scale-out) một cách nhanh chóng bằng cách thêm IP vào inventory.

- Về khả năng quản trị: Giao diện Horizon trực quan, hỗ trợ đầy đủ các thao tác quản lý máy chủ ảo, mạng và lưu trữ mà không cần can thiệp sâu vào dòng lệnh.

Từ nền tảng Private Cloud cơ bản đã xây dựng, đề tài có thể mở rộng theo các hướng sau:

- Xây dựng cụm High Availability (HA): Triển khai mô hình 3 Controller Nodes kết hợp với Keepalived và HAProxy để đảm bảo hệ thống vẫn hoạt động ngay cả khi một Node điều khiển bị lỗi.

- Tích hợp Container Orchestration: Sử dụng dự án OpenStack Magnum để cung cấp dịch vụ Kubernetes trên nền tảng OpenStack, giúp quản lý cả Virtual Machines và Containers trên cùng một hạ tầng.

- Tối ưu hóa lưu trữ với Ceph: Thay thế LVM bằng cụm lưu trữ phân tán Ceph để tăng khả năng chịu lỗi và hiệu năng đọc/ghi cho Volume và Image.

- Triển khai Monitoring: Tích hợp bộ công cụ Prometheus & Grafana để theo dõi biểu đồ tải của CPU, RAM, Network của toàn bộ hệ thống theo thời gian thực.

TÀI LIỆU THAM KHẢO

Giáo trình & Sách

[1] S. S. Nath, Mastering OpenStack: Design, deploy, and manage a scalable OpenStack infrastructure, 2nd ed., Birmingham, UK: Packt Publishing, 2017.

[2] O. Kallo, OpenStack Administration with Ansible 2: Learn how to automate your OpenStack cloud using the most popular IT automation tool, 1st ed., Birmingham, UK: Packt Publishing, 2016.

[3] V. Jalali, OpenStack Cloud Computing Cookbook: Over 110 recipes to design, develop, and deploy a robust OpenStack cloud, 4th ed., Birmingham, UK: Packt Publishing, 2018.

[4] J. Denton, Learning OpenStack: Design, deploy, and manage a scalable OpenStack infrastructure, 1st ed., Birmingham, UK: Packt Publishing, 2015.

Website

[5] “OpenStack Caracal (2024.1) Documentation,” OpenStack Foundation, [Online].

Available: <https://docs.openstack.org/caracal/>. [Accessed: October 15, 2025].

[6] “Kolla-Ansible: Deployment of OpenStack services in Docker containers,” OpenStack Foundation, [Online].

Available: <https://docs.openstack.org/kolla-ansible/latest/>. [Accessed: October 15, 2025].

[7] “Ansible Documentation: Installation and User Guide,” Red Hat, Inc., [Online].

Available: <https://docs.ansible.com/>. [Accessed: October 16, 2025].

[8] “Docker Documentation: Accelerate your development with Docker,” Docker, Inc., [Online].

Available: <https://docs.docker.com/>. [Accessed: October 16, 2025].

[9] “Ubuntu 24.04 LTS (Noble Numbat) Release Notes,” Canonical Ltd., [Online].

Available: <https://discourse.ubuntu.com/t/noble-numbat-release-notes/>. [Accessed: October 17, 2025].